

UNIVERSITY OF MORATUWA

Faculty of Information Technology

IN 3001 Industrial Training

TRAINING REPORT



Tishanth Sivakumar

224199T

Matrix Mantra (Pvt) Ltd

02 February 2026 – 31 July 2026

Submitted on: 19th August 2026

Preface

Industrial training is an essential component of the Bachelor of Science Honours Degree in Information Technology programme offered by the University of Moratuwa. It provides undergraduates with the opportunity to bridge the gap between academic learning and professional practice by gaining hands-on experience in a real-world working environment. This report has been prepared as a requirement for the successful completion of the Industrial Training programme and documents my internship experience at Matrix Mantra (Pvt) Ltd. from 02 February 2026 to 31 July 2026.

The primary purpose of this report is to present a comprehensive account of the knowledge, technical skills, and professional competencies that I developed during the six-month training period. Throughout the internship, I was exposed to industry-standard software engineering practices, collaborative development environments, modern technologies, and professional workplace ethics. Working alongside experienced software engineers enabled me to strengthen my understanding of software development methodologies while applying the theoretical concepts learned throughout my academic studies to practical software solutions.

This report provides an overview of Matrix Mantra (Pvt) Ltd., including its background, organizational structure, business operations, services, and technology-driven environment. It further describes the responsibilities assigned to me, the projects and technical tasks I was involved in, the tools and technologies utilized, the challenges encountered during the internship, and the approaches adopted to overcome them. In addition, the report reflects on the valuable technical knowledge, problem-solving abilities, communication skills, teamwork, and professional practices that were developed during the training period.

The internship experience significantly enhanced my confidence as an aspiring software engineering professional by providing practical exposure to real industry expectations and software development processes. It also strengthened my understanding of how software engineering principles are applied within a professional organization to deliver reliable and high-quality software solutions. I believe that the knowledge and experience gained during this training will serve as a strong foundation for my future academic pursuits and professional career in the field of Information Technology.

Acknowledgement

First and foremost, I wish to express my deepest gratitude to Prof. (Mrs.) Thanuja Sandanayake, Dean of the Faculty of Information Technology, University of Moratuwa, and Mrs. Ashika Fernando, Training Engineer of the Faculty of Information Technology, for their continuous coordination, invaluable guidance, and structured support throughout the industrial training placement process. Their unwavering commitment to bridging our academic curriculum with professional industry expectations has been instrumental in securing this valuable learning opportunity and ensuring the successful completion of my training module.

I extend my sincere appreciation to Matrix Mantra (Pvt) Ltd. for granting me the opportunity to join their engineering team as a Trainee Software Engineer. I am profoundly grateful to Mr. Mithun Paresith, Managing Representative and Officer in Charge at Matrix Mantra (Pvt) Ltd., for welcoming me into the company's collaborative workspace and allowing me to contribute to global-standard commercial solutions.

I owe a special note of gratitude to my Project Manager, Technical Lead, and Supervisor, Mr. Pradeep. His tireless mentorship, constructive code reviews, and architectural insights throughout the redevelopment of the LuckyMobile.lk storefront and the foundational development of the multi-tenant MantraOne SaaS platform have been critical to my professional growth. His patience and technical expertise in navigating complex state management, database schema design, and server-side logic have greatly enhanced my practical problem-solving skills and technical confidence.

I would also like to express my heartfelt thanks to my fellow developers and team members at Matrix Mantra (Pvt) Ltd. for their constant collaboration, supportive feedback, and shared expertise during my remote internship. The collaborative discussions within our development pipeline significantly contributed to my personal and technical progress. Finally, I am deeply thankful to my family, academic mentors, and friends for their constant encouragement, understanding, and support, which motivated me to navigate the technical challenges of this training period and successfully compile this industrial training final report.

Table of Contents

Preface.....	i
Acknowledgement	ii
Table of Contents.....	iii
List of Figures	iv
List of Tables	v
Chapter 1 – Introduction to the Training Establishment.....	1
1.1 Overview, History, and Operational Framework.....	1
1.1.1 Corporate Overview and Strategic Vision	1
1.1.2 Historical Development and Technical Evolution.....	1
1.1.3 Dual-Office Operational Framework.....	2
1.2 Corporate Business Domains and Delivery Methodologies	2
1.2.1 Core Business Domains and Technology Toolkits.....	2
1.2.2 Agile Project Lifecycle (Movmi Case Study).....	3
1.3 Organizational Structure and Hierarchical Levels	4
1.3.1 Executive Leadership and Sri Lankan Division Governance	4
1.3.2 Technical and Project Management Hierarchy	5
1.3.3 Divisional Classification of Technical Teams	6
1.4 Present Performance and Global Scale Indicators	6
1.5 SWOT Analysis (Strengths, Weaknesses, Opportunities, Threats).....	7
1.5.1 Internal Strengths of Matrix Mantra	7
1.5.2 Internal Weaknesses of Matrix Mantra.....	8
1.5.3 External Opportunities for Market Expansion.....	8
1.5.4 External Threats and Macroeconomic Challenges	9
1.6 Usefulness of the Training Establishment to Sri Lankan Society.....	9
1.6.1 Macroeconomic Contributions and Foreign Revenue Inflow.....	10
1.6.2 Northern Province Tech Empowerment and Local Capacity Building	10
1.6.3 Academic Partnerships and Strategic Knowledge Transfer.....	10
1.7 Strategic Suggestions for Growth and Identified Constraints.....	12
1.7.1 Technical Bottlenecks and Governance Suggestions	12
1.7.2 Standardizing Trainee Onboarding Pipelines	13
1.7.3 Sector Diversification and Local Socio-Technical Integration.....	13

Chapter 2 – Training Experiences.....	14
2.1 Work Environment, Project Allocation, and Workplace Adaptations.....	14
2.1.1 Worksite, Departments, and Key Training Personnel	14
2.1.2 Technical Onboarding, Workspace Setup, and Initial Tools Provisioning.....	15
2.1.3 Corporate Induction and Strategic Adaptations to Hybrid Workspaces.....	15
2.2 The Foundational Learning Path and Preparatory Engineering Research	16
2.2.1 Technical Onboarding, Self-Directed Learning, and Documentation Studies	16
2.2.2 E-Commerce System Analysis and Requirements	17
2.3 Project 1: Complete Redevelopment of the LuckyMobile.lk Storefront	17
2.3.1 E-Commerce Usability Analysis and Requirements Elicitation.....	18
2.3.2 Relational Database Modeling and Prisma ORM Schema Design.....	18
2.3.3 Layered Express.js Backend: Routes, Controllers, and Services.....	19
2.3.4 Secure Administrative Authentication using JWT and bcrypt.....	20
2.3.5 Frontend Application Architecture with Next.js and Tailwind CSS	20
2.3.6 Dynamic Variant Selection and WhatsApp Direct Ordering.....	22
2.3.7 Overlap-Validated Promotion Scheduler and Admin Control Panel.....	23
2.3.8 Storefront Performance Optimization and Responsive Interface Validation	24
2.4 Project 2: MantraOne Multi-Tenant SaaS E-Commerce Platform	25
2.4.1 Software Architectural Principles of Shared-Database Multi-Tenancy.....	25
2.4.2 Centralized Storefront Router and Tenant Resolution Middleware	26
2.4.3 Scoped Database Migrations and Prisma Transaction Integrity	27
2.4.4 Storefront Branding Engine and CSS Variables Theme Resolution	28
2.4.5 Integrating AWS S3 for Private Multi-Tenant Media Storage	29
2.4.6 End-to-End Order Processing and Atomic Inventory Synchronization.....	30
2.4.7 Security Audit: Direct URL Access Bypass and Custom Route Guards.....	30
2.4.8 Event-Driven Real-Time Order Alerts using Socket.IO.....	31
2.4.9 Automating Transactional Emails with Scoped Nodemailer Services	32
2.4.10 Storefront Search Optimization and State-Preserving Product Filters	33
2.5 Industry Processes, Office Practices, and Technical Tooling Specifications	33
2.5.1 Professional Production Processes and Quality Assurance	33
2.5.2 Administrative, Office, and Collaborative Workspace Practices	34
2.5.3 Financial Procedures and Commercial Client Operations	34
2.5.4 Technical Equipment, Developer Tooling, and Environmental Specifications.....	35

2.5.5 Occupational Health and Safety (OHS) and Office Safety Procedures	35
2.6 Reflective Analysis of Technical Hurdles, Personal Gains, and Weaknesses	36
2.6.1 Technical Challenges, Work-Related Difficulties, and Implemented Solutions	36
2.6.2 Communication and Coordination Difficulties in Hybrid Workspaces.....	37
2.6.3 Evaluation of Professional and Interpersonal Gains	37
2.6.4 Honest Identification of Personal Weaknesses and Areas for Growth	38
2.6.5 Perspective on Skills and Experiences Not Gained During the Placement	39
 Chapter 3 – Conclusion and Recommendations	 40
3.1 Synthesis and Summary of the Industrial Training Experience.....	40
3.2 Critique of the Training Establishment (Matrix Mantra).....	41
3.2.1 Evaluation of Mentorship, Training Personnel, and Work Environment	41
3.2.2 Identified Deficiencies and Suggestions for Organizational Improvement.....	41
3.3 Critique of the University and NAITA Industrial Training Programme	42
3.3.1. Evaluation of University Academic Curriculum and Relevancy.....	42
3.3.2. Evaluation of NAITA Coordinated Administration and Monitoring Processes.....	43
3.4 Expectations Met, Personal Growth, and Professional Gains.....	44
3.5 Actionable Recommendations for Future Trainees	45

List of Figures

Figure 1.1 Matrix Mantra Official Corporate Logo	1
Figure 1.2 Agile SDLC at Matrix Mantra.....	4
Figure 1.3 Mr. Mithun Paresith (Founder & CEO)	5
Figure 1.4 Matrix Mantra Organizational Structure	6
Figure 1.5 Matrix Mantra–SLIIT Northern Uni MOU Signing Ceremony	11
Figure 1.6 MERN Stack Technical Workshop for SLIIT Students	12
Figure 1.7 Redex Tech Team Brainstorming Session	12
Figure 2.1 Prisma Configuration File	19
Figure 2.2 JWT Authentication Response	20
Figure 2.3 Storefront Homepage.....	21
Figure 2.4 Product Detail Page	23
Figure 2.5 Admin Dashboard.....	24
Figure 2.6 Authentication API Endpoints.....	27
Figure 2.7 Settings API Endpoints	29
Figure 2.8 Product Creation Form	30
Figure 2.9 Customer Management Module	31
Figure 2.10 Notification API Endpoints	32

List of Tables

Table 1.1 Matrix Mantra Service Pillars and Technology Stack	3
Table 1.2 Matrix Mantra Performance Metrics	7
Table 2.1 Multi-Tenant Database Isolation Strategy	28

CHAPTER 1

INTRODUCTION TO THE TRAINING ESTABLISHMENT

1.1. Overview, History, and Operational Framework

1.1.1. Corporate Overview and Strategic Vision

Matrix Mantra (Pvt) Ltd is a leading global software product engineering and Information Technology services organization specializing in high-performance web development, mobile application engineering, custom artificial intelligence (AI) solutions, user interface and user experience (UI/UX) design, and quality assurance (QA) outsourcing. Established to deliver world-class digital transformation solutions, the company operates as a trusted technology partner for startups, mid-sized businesses, and global enterprises across international markets, including Australia, Europe, the Middle East, and North America. By integrating design thinking, modern technology stacks, and data-driven business strategies, Matrix Mantra helps organizations build resilient digital foundations and optimize complex operational workflows.



Figure 1.1 Matrix Mantra Official Corporate Logo

1.1.2. Historical Development and Technical Evolution

Since its founding, Matrix Mantra has transitioned from a boutique web design agency into a highly specialized, multi-vertical IT product engineering firm. To meet the demands of international clients for secure, interactive, and high-performance cloud applications, the organization systematically shifted its focus toward advanced, enterprise-grade software engineering.

The company transitioned its primary development environment to modern full-stack technologies, adopting Next.js, React, Node.js, and Express for web development, and Flutter and React Native for cross-platform mobile engineering. On the database layer, the company

adopted PostgreSQL managed through Prisma Object-Relational Mapping (ORM) to enforce strict schema-validation rules and type safety. This technical evolution enabled the company to participate in high-value foreign projects, successfully transitioning from basic content management systems to complex, multi-tenant Software-as-a-Service (SaaS) architectures, real-time database management systems, and custom AI implementations.

1.1.3. Dual-Office Operational Framework

To balance high-intensity offshore development capabilities with localized onshore client relations, Matrix Mantra operates a coordinated dual-office framework:

- **Jaffna Development Center (Sri Lanka):** Located at No. 77, Ramanathan Road, Jaffna, Sri Lanka, this offshore facility functions as the technological core of the company. It houses the primary engineering, UI/UX, project management, and QA teams who collaborate to architect, implement, and maintain the company's entire software portfolio, including the LuckyMobile.lk e-commerce storefront and the multi-tenant MantraOne SaaS system.
- **Victoria Corporate Office (Australia):** Situated at The Parkway, Hampton Park, Victoria (VIC) 3976, Australia, this onshore division drives international business consulting, handles client onboarding, manages regional accounts, and ensures compliance with global market standards.

1.2. Corporate Business Domains and Delivery Methodologies

1.2.1. Core Business Domains and Technology Toolkits

Matrix Mantra delivers integrated, end-to-end digital solutions by combining creative design, robust engineering, and data-driven strategy across diverse industrial domains:

Table 1.1 Matrix Mantra Service Pillars and Technology Stack

Service Pillar	Primary Functions and Objectives	Associated Software Stacks and Toolkits
Digital Strategy	Business transformation, workflow analysis, digital process optimization, and data pipeline development.	Google Analytics, Power BI, HubSpot CRM, Notion
UI/UX Design	User research, user experience design, wireframing, prototyping, and interactive interface design.	Figma, Adobe XD, Framer, Adobe Creative Cloud

Service Pillar	Primary Functions and Objectives	Associated Software Stacks and Toolkits
Web Development	Development of responsive web applications, frontend and backend systems, RESTful APIs, and serverless web solutions.	Next.js, React.js, Node.js, Tailwind CSS, PostgreSQL
Mobile Engineering	Development of cross-platform mobile applications with offline synchronization and real-time communication capabilities.	Flutter, React Native, Firebase, Kotlin, Swift
Artificial Intelligence	Development of intelligent automation systems, generative AI applications, machine learning models, and predictive analytics solutions.	OpenAI API, TensorFlow, LangChain, Python Frameworks
Digital Marketing	Search engine optimization (SEO), digital advertising, social media campaign management, website analytics, and lead generation.	Meta Business Suite, Google Ads, Google Search Console, Google Analytics 4
Quality Assurance	Software quality assurance through automated testing, REST API validation, defect tracking, and agile sprint management.	Selenium, Postman, Jira, Cypress Testing Framework

1.2.2. Agile Project Lifecycle (Movmi Case Study)

To consistently deliver global-standard software products, Matrix Mantra strictly adheres to an iterative Agile Scrum methodology. This approach divides complex project scopes into manageable one-to-two-week development cycles (sprints), bounded by core Scrum ceremonies: Sprint Planning, Daily Standups, Sprint Reviews, and Sprint Retrospectives.

The practical execution of this methodology is demonstrated through the development lifecycle of the Movmi platform—a comprehensive shipping and supply chain management system developed for enterprises in Saudi Arabia. The implementation of this platform followed a rigorous, four-phase engineering pipeline:

1. **Phase I: Discovery and Requirements Elicitation:** The onshore business consulting team in Australia collaborated with the client to map complex supply chain workflows and Saudi Arabian customs clearance protocols. These business processes were translated into technical specifications, database schema flowcharts, and REST API contract designs, which were modularized as epics in Jira.
2. **Phase II: High-Fidelity UI/UX Prototyping:** The UI/UX division created dynamic, high-fidelity prototypes in Figma capturing critical user paths—such as automated container tracking pages and customs documentation upload panels—undergoing active client review rounds before development.
3. **Phase III: Full-Stack Software Engineering:** Operating within a unified Git monorepo structure, backend engineers built a scalable, microservices-oriented

architecture using Node.js and Express, communicating with PostgreSQL through Prisma ORM to ensure complete transactional integrity. Simultaneously, frontend engineers utilized Next.js and React, styled with utility-first Tailwind CSS, to build a highly responsive, component-driven tracking dashboard.

4. **Phase IV: Integration, Quality Assurance, and Deployment:** The QA team conducted automated end-to-end testing using Cypress and Selenium to validate user workflows, alongside API testing via Postman. Once validated, the code was deployed via automated CI/CD pipelines to secure cloud infrastructures, utilizing international CDNs to guarantee sub-second page loading times for logistics managers in Saudi Arabia.

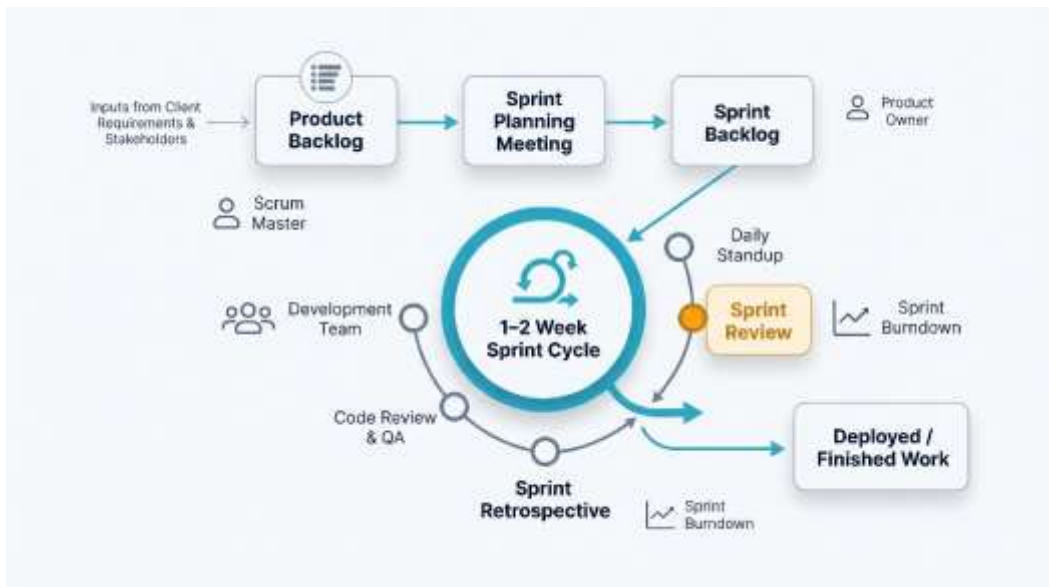


Figure 1.2 Agile SDLC at Matrix Mantra

1.3. Organizational Structure and Hierarchical Levels

1.3.1. Executive Leadership and Sri Lankan Division Governance

Matrix Mantra utilizes a highly collaborative, streamlined operational structure designed to optimize productivity, maintain agile adaptability, and ensure rapid, high-quality delivery across global client portfolios. At the regional executive level, the Sri Lankan offshore development division is governed directly under the leadership of Mr. Mithun Paresith, who functions as the Managing Representative and Officer in Charge. Mr. Mithun Paresith oversees the operational administration, local infrastructure management, academic partnerships, legal compliance, and strategic resource allocation of the Jaffna center. By coordinating directly with

the onshore corporate office in Victoria, Australia, the regional executive management ensures that offshore engineering outputs remain aligned with global expectations and SLA commitments.



Figure 1.3 Mr. Mithun Paresith (Founder & CEO)

1.3.2. Technical and Project Management Hierarchy

Directly reporting to regional executive management is the engineering and project management division, which guides the software development lifecycle from early discovery phases through to production releases. In the development workspace, software projects—such as the e-commerce redevelopment of the LuckyMobile.lk storefront and the multi-tenant architecture of the MantraOne SaaS platform—are managed under the close supervision of Mr. Pradeep, who acts as the Project Manager, Technical Lead, and Primary Supervisor.

Mr. Pradeep is responsible for defining system architectures, formulating API contracts, auditing database relational schemas, conducting peer code reviews, and enforcing industry-standard software engineering practices. Trainee Software Engineers and interns report directly to Mr. Pradeep, receiving continuous technical guidance, architectural instruction, and daily professional mentorship.

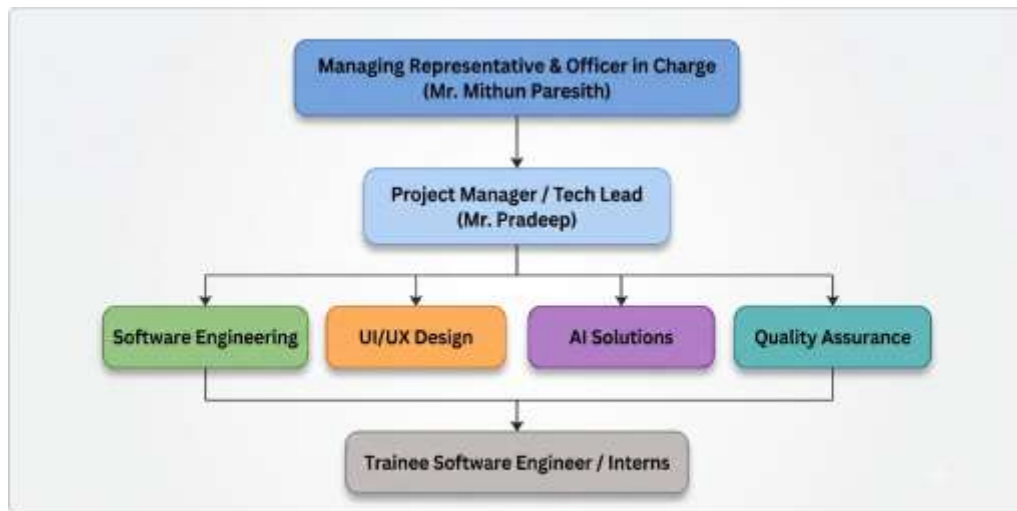


Figure 1.4 Matrix Mantra Organizational Structure

1.3.3. Divisional Classification of Technical Teams

The operational backbone of Matrix Mantra is divided into distinct, highly specialized technical divisions to ensure domain expertise is applied directly to each phase of a project:

1. **Software Product Engineering Division:** Consisting of senior developers, software engineers, and trainee engineers, this division is responsible for backend and frontend codebase implementation using Next.js, Express, Prisma ORM, and PostgreSQL.
2. **User Interface & Experience (UI/UX) Division:** This creative stream is dedicated to user research, wireframing, high-fidelity dynamic prototyping, and design system creation using Figma and Adobe XD.
3. **AI Solutions & Research Division:** Operating as an innovation hub, this team builds generative workflows, custom machine learning classifications, and predictive data-driven models using TensorFlow, LangChain, and OpenAI APIs.
4. **Quality Assurance & Testing Division:** This independent division ensures that final software builds are reliable, performant, and secure before public deployment, utilizing Cypress, Selenium, and Postman.

1.4. Present Performance and Global Scale Indicators

Matrix Mantra has established a strong commercial footprint within the global software export ecosystem, positioning itself as a reliable product engineering partner for international enterprises. The company's present performance is characterized by its ability to scale diverse

corporate brands across highly competitive global markets. Rather than operating as a localized IT service provider, the company actively exports high-performance digital products to clients situated across Australia, Europe, the Middle East, and North America.

To date, the company has successfully scaled over 120 brands globally, delivered more than 50 major technical projects, and achieved an outstanding client satisfaction score of 9.8 out of 10. This success is paired with a 98% client retention rate, maintained through structured post-completion support, including ongoing security updates, real-time server performance monitoring, and continuous cloud optimization.

From a financial perspective, the continuous inflow of foreign exchange from these high-value international B2B software exports ensures the company's robust financial stability and consistent commercial profitability. This strong financial performance allows Matrix Mantra to operate without relying on external local credit, fund continuous research and development in its AI Solutions division, and maintain stable employment and upskilling opportunities in the Northern Province despite macroeconomic volatility in Sri Lanka.

Table 1.2 Matrix Mantra Performance Metrics

Metric Category	Baseline Value	Primary Engineering Driver	Strategic Business Impact
Global Brands Scaled	120+ Brands	Next.js, SEO optimization, multi-region CDN, localized user interfaces	Expanded global market reach across APAC, Europe, and the Middle East.
Successful Projects	50+ Projects	Next.js, Node.js REST APIs, Prisma ORM, PostgreSQL	Demonstrated expertise in delivering solutions across logistics, e-commerce, and education sectors.
Client Satisfaction	9.6 / 10.0	Agile Scrum, sprint reviews, Jira collaboration	Improved customer satisfaction, resulting in stronger reputation and client referrals.
Client Retention	98% Retention	Post-deployment support, SLA maintenance, cloud optimization	Strengthened long-term client relationships and generated recurring business opportunities.

1.5. SWOT Analysis (Strengths, Weaknesses, Opportunities, Threats)

1.5.1. Internal Strengths of Matrix Mantra

The competitive positioning of Matrix Mantra (Pvt) Ltd. within the global software export market is underpinned by several core internal strengths. Foremost among these is the

company's commitment to a modern, highly scalable, and standardized technical stack. By standardizing its development pipelines on Next.js, React.js, Express.js, Prisma ORM, and PostgreSQL, the company ensures that its engineering teams can rapidly build, optimize, and maintain complex web and enterprise applications. This modern architecture allows the organization to deliver exceptional performance, sub-second page rendering, and built-in search engine optimization, which are critical requirements for global commercial platforms.

Additionally, the company's dual-office operational framework represents a significant structural strength. By maintaining an onshore consulting and business development office in Victoria, Australia, alongside a high-density offshore engineering center in Jaffna, Sri Lanka, Matrix Mantra successfully bridges the gap between international client relations and cost-effective product delivery. This setup is supported by active academic-industry collaborations, such as the company's formal Memorandum of Understanding (MOU) and specialized MERN stack training workshops with SLIIT Northern Uni. These initiatives establish a direct pipeline for recruiting top-tier technical talent in the Northern Province, ensuring continuous organizational growth and a sustainable competitive edge.

1.5.2. Internal Weaknesses of Matrix Mantra

Despite its strong technical capabilities, certain internal operational limitations can affect the company's long-term scalability. A primary weakness is the high concentration of project management, technical oversight, and architectural decision-making within a few key supervisory roles. For instance, critical full-stack platforms and SaaS engines—including the LuckyMobile.lk storefront redevelopment and the multi-tenant MantraOne system—rely heavily on the continuous guidance and code-review capacity of Mr. Pradeep, who acts as Project Manager, Technical Lead, and Primary Supervisor. This centralized oversight structure can create operational bottlenecks as the number of parallel global projects increases.

Another internal constraint is the steep learning curve and onboarding complexity associated with the company's advanced codebases. Because Matrix Mantra utilizes unified Git monorepos, Prisma transaction layers, and complex multi-tenant database isolations, incoming Trainee Software Engineers and junior developers often face initial difficulties in adapting to the development environment. Furthermore, while the company's remote-first and hybrid work model offers operational flexibility, it requires exceptional self-discipline and rigorous virtual

communication from developers. This setup can occasionally lead to minor coordination friction compared to fully centralized, on-site engineering pipelines.

1.5.3. External Opportunities for Market Expansion

Matrix Mantra is well-positioned to capitalize on several emerging trends in the global technology sector. The rapid commercial adoption of Artificial Intelligence and intelligent workflow automation represents a massive growth opportunity. By expanding its dedicated AI Solutions division—which currently integrates OpenAI APIs, TensorFlow, and LangChain frameworks—the company can design and upsell custom machine learning integrations, predictive analytics dashboards, and conversational AI assistants to its existing portfolio of over 120 brands.

Furthermore, the expansion of global e-commerce, third-party logistics, and digital matchmaking ecosystems offers high-value opportunities to deploy specialized software products. Building on the successful deployment of international enterprise platforms like Movmi in Saudi Arabia, the company can package its core multi-tenant SaaS capabilities into proprietary, subscription-based software products. Locally, the ongoing digital transformation within Sri Lanka’s Northern Province opens opportunities to partner with local government authorities, educational institutions, and regional businesses, establishing Matrix Mantra as a central driver of regional economic modernization.

1.5.4. External Threats and Macroeconomic Challenges

The external business environment presents several systemic threats that Matrix Mantra must navigate to maintain its operational stability. The most critical threat facing the Sri Lankan IT sector is the high rate of professional brain drain. Economic volatility and inflation have driven many highly skilled senior developers and system architects to seek employment opportunities abroad. This migration makes the retention of local technical expertise and the maintenance of senior engineering roles highly challenging.

Additionally, the global software development market is intensely competitive, with established multinational product engineering firms in Eastern Europe, India, and Southeast Asia vying for the same client base. To maintain its high 98% client retention rate and 9.6

satisfaction score, Matrix Mantra must continuously invest in research, development, and upskilling, which requires significant capital allocation. Finally, regional infrastructure constraints, such as unexpected power interruptions or internet bandwidth fluctuations in localized areas, can occasionally disrupt remote development workflows, requiring constant adjustments to maintain strict delivery schedules.

1.6. Usefulness of the Training Establishment to Sri Lankan Society

1.6.1. Macroeconomic Contributions and Foreign Revenue Inflow

Matrix Mantra (Pvt) Ltd plays a significant role in supporting the Sri Lankan economy during a period of macroeconomic challenges and foreign exchange instability. Operating as a high-value exporter of digital products and custom software solutions, the company systematically channels foreign currency into the country from international markets, including Australia, Europe, the Middle East, and North America. Because software engineering exports require minimal material import costs, the company's export revenue is highly beneficial for the nation's balance of payments, helping stabilize the domestic currency and contributing directly to national recovery.

1.6.2. Northern Province Tech Empowerment and Local Capacity Building

One of the most valuable social impacts of Matrix Mantra is its dedication to decentralizing the Sri Lankan IT industry. Historically, high-density IT infrastructure, career opportunities, and software engineering headquarters have been concentrated almost exclusively within the Western Province. By establishing its offshore software delivery and artificial intelligence engineering center at No. 77, Ramanathan Road, Jaffna, Matrix Mantra has helped build a thriving technology hub in the Northern Province. This regional presence provides high-paying, global-standard career pathways for northern youth and engineering graduates, allowing them to secure competitive software development roles without having to migrate, playing an active role in regional talent retention and reducing local unemployment.

1.6.3. Academic Partnerships and Strategic Knowledge Transfer

The company actively collaborates with academic institutions to bridge the gap between

undergraduate curricula and the evolving demands of the global software industry. This commitment is highlighted by the formal Memorandum of Understanding (MOU) signed between Matrix Mantra and SLIIT Northern Uni in Jaffna. This partnership facilitates ongoing knowledge transfer, internships, and professional mentoring programs for undergraduates. To ensure that student developers gain hands-on experience with modern, industry-standard software stacks, the company conducts intensive technical workshops, such as its specialized MERN stack development sessions for SLIIT students.

Furthermore, the organization organizes Redex Tech Team Brainstorming — Innovation in Action sessions. These are collaborative discussions and technical brainstorming events hosted by the offshore engineering division. In these sessions, senior developers, QA specialists, and trainee engineers meet to discuss emerging technology trends, evaluate open-source toolkits, and brainstorm innovative architectures for ongoing SaaS developments. This active research-and-development environment ensures that local engineers are constantly upskilled, exposed to high-level computer science concepts, and trained to implement next-generation commercial solutions.



Figure 1.5 Matrix Mantra–SLIIT Northern Uni MOU Signing Ceremony



Figure 1.6 MERN Stack Technical Workshop for SLIIT Students



Figure 1.7 Redex Tech Team Brainstorming Session

1.7. Strategic Suggestions for Growth and Identified Constraints

1.7.1. Technical Bottlenecks and Governance Suggestions

To support its rapid expansion, Matrix Mantra must address certain internal operational constraints. A primary challenge is the company's reliance on a limited number of senior technical leaders for architectural finalization, database schema auditing, and code reviews. Currently, complex development pipelines—including the storefront redevelopment of LuckyMobile.lk and the multi-tenant architecture of the MantraOne SaaS platform—rely heavily on the continuous oversight of the Technical Lead and Supervisor, Mr. Pradeep.

As the company scales its client base beyond its current 120 brands, this centralized governance structure may create operational bottlenecks. To mitigate this risk, Matrix Mantra should implement a more decentralized technical hierarchy. By establishing intermediate technical lead positions and empowering experienced Associate Software Engineers to conduct preliminary code reviews and coordinate baseline sprints within Jira, the company can accelerate delivery speeds and scale its operations more effectively.

1.7.2. Standardizing Trainee Onboarding Pipelines

Another constraint identified within the development environment is the steep learning curve that incoming Trainee Software Engineers and junior developers face when first working with the company's complex production codebases. Because Matrix Mantra utilizes advanced, unified Git monorepos, Prisma transaction layers, and intricate multi-tenant database systems, new developers often require a long transition period before they can contribute effectively to commercial features.

To streamline this onboarding process, the company should invest in building comprehensive, standardized internal technical documentation. Creating dedicated, step-by-step developer guides—such as monorepo setup instructions, Prisma PostgreSQL migration steps, API endpoint definitions, and testing checklists—would significantly reduce the onboarding period. Additionally, assigning new trainees a dedicated peer mentor during their first month would help them adapt smoothly to the development environment, reducing initial friction and allowing them to deliver commercial-grade features more quickly.

1.7.3. Sector Diversification and Local Socio-Technical Integration

While Matrix Mantra has achieved outstanding commercial success by exporting software products to international clients, the company has an opportunity to diversify its portfolio by partnering with local Sri Lankan organizations. The company's expertise in web development, mobile applications, and AI integrations could be used to build tailored digital solutions for regional schools, healthcare facilities, cooperative societies, and local government bodies in the Northern Province.

By designing accessible, localized software—such as digital administrative platforms,

inventory management systems for local businesses, or online educational portals—Matrix Mantra can directly support the digitalization of northern society. This local integration would not only expand the company's regional brand visibility but also demonstrate its capabilities as a key driver of community development, complementing its successful international software export model.

CHAPTER 2

TRAINING EXPERIENCES

2.1. Work Environment, Projects Allocation, and Workplace Adaptations

2.1.1. Worksite, Departments, and Key Training Personnel

The twenty-four-week industrial training placement was completed at Matrix Mantra (Pvt) Ltd within the Software Product Engineering Division, operating from February 2, 2026, to July 31, 2026. The training was structured to expose the trainee to the complete lifecycle of professional software products, transitioning from frontend interfaces to complex, secure backend microservices. While the trainee worked primarily from a remote development setup to align with the company's hybrid operational model, the offshore delivery operations were centrally coordinated at the company's development center located at No. 77, Ramanathan Road, Jaffna, Sri Lanka.

Throughout the internship, the trainee's progress, technical direction, and professional development were closely monitored by senior engineering leaders. Operational administration, local resource allocation, and academic-industry partnership guidelines were directed by Mr. Mithun Paresith, who functioned as the Managing Representative and Officer-in-Charge of the Jaffna center. Direct technical supervision, architectural guidance, sprint planning, and daily code reviews were conducted by Mr. Pradeep, who acted as the Project Manager, Technical Lead, and Primary Internship Supervisor. Under this structured mentorship model, the trainee's industrial training was divided into two distinct, high-impact project phases:

- **Phase I:** LuckyMobile.lk Storefront Redevelopment (Weeks 1 to 8): Dedicated to the complete full-stack re-engineering of an active e-commerce and retail electronics platform.

- **Phase II:** MantraOne Multi-Tenant SaaS E-Commerce Platform (Weeks 9 to 24): Focused on the design, modular backend implementation, and security auditing of a shared-database, multi-tenant Software-as-a-Service system.

2.1.2. Technical Onboarding, Workspace Setup, and Initial Tools Provisioning

The onboarding process began with the provisioning of the local developer workspace and security access configurations on February 2, 2026. To support the technical demands of full-stack engineering and local database execution, the company provided a standardized hardware and software environment. The trainee was equipped with a DELL Latitude 3490 laptop, featuring an Intel Core i5 8th Generation processor, 16 GB of Random Access Memory (RAM), and a 64-bit operating system, paired with A4Tech HS-30i on-ear headphones to coordinate during virtual team syncs.

The initial workspace configuration required installing the core runtime environments and developer tools. The trainee installed Node.js as the primary JavaScript runtime, configured Git for distributed version control, and set up a local PostgreSQL instance as the relational database engine. To simplify database management and query execution, the trainee installed DBeaver as a database administration client.

Following the environment configuration, the trainee was granted secure developer credentials to the company's GitHub organization and Jira workspaces, allowing them to clone the centralized repositories, study coding style guidelines, and verify that the local compilers and build engines completed without errors.

2.1.3. Corporate Induction and Strategic Adaptations to Hybrid Workspaces

To integrate the trainee into the professional culture of the organization, the human resources team conducted a detailed corporate induction. This session focused on corporate ethics, intellectual property ownership, data protection compliance, and secure data handling. Because Matrix Mantra engineers commercial software for international clients, maintaining strict database security and safeguarding proprietary business logic is critical. The trainee completed a detailed security orientation, signing standard non-disclosure agreements (NDAs) to protect client confidentiality and prevent unauthorized sharing of platform architectures.

Adapting to the hybrid and remote working model required the trainee to develop strong self-discipline and professional communication skills. To prevent the isolation and coordination delays often associated with remote work, the development team utilized structured communication protocols. Daily tasks, technical queries, and architectural updates were synchronized through dedicated channels on Slack, while longer system design discussions, weekly sprint reviews, and technical demonstrations were conducted via Zoom.

This organized setup taught the trainee how to manage technical blockers proactively, document progress clearly, and coordinate with cross-functional team members, including project managers, UI/UX designers, and QA specialists.

2.2. The Foundational Learning Path and Preparatory Engineering Research

2.2.1. Technical Onboarding, Self-Directed Learning, and Documentation Studies

During the first week of the industrial placement, Mr. Pradeep established a structured, self-directed learning path to prepare the trainee for full-stack engineering tasks within the company's standard tech stack. Although the trainee possessed foundational knowledge of web development, transitioning to enterprise-grade software products required mastering advanced full-stack concepts, including server-side rendering, type safety, relational database optimization, and secure API design.

The learning path focused on several core technologies:

- **Next.js (App Router):** Studying server-side rendering (SSR), dynamic routing, page optimization, and frontend rendering modes.
- **Prisma Object-Relational Mapping (ORM):** Understanding type-safe query building, database relationships, and migration workflows.
- **PostgreSQL:** Exploring database normalization, index optimizations, concurrent query execution, and database constraints.
- **Express.js Middleware:** Learning request validation pipelines, session management, and role-based route protection.

The trainee spent the initial week studying official framework documentations, completing

practical coding exercises, and building local test applications to verify their understanding of full-stack data flows.

2.2.2. E-Commerce System Analysis and Requirements

Alongside technical learning, the supervisor assigned the trainee to perform a detailed requirements elicitation and usability audit of the legacy LuckyMobile.lk website. This system discovery process was designed to teach the trainee how real-world commercial requirements are gathered and analyzed. The trainee analyzed the old site's performance, identifying several critical usability bottlenecks:

- **High Database Latency:** Repetitive database queries during catalog rendering caused page load times exceeding four seconds.
- **Complicated Checkout Flow:** A multi-step checkout form created high transaction friction, leading to significant cart abandonment.
- **Mobile Responsiveness Issues:** The user interface did not scale correctly on mobile devices, resulting in misaligned layouts on smaller screens.

The trainee compiled these findings into a detailed functional requirements specification document. This document separated system requirements into two distinct streams: the public, high-speed storefront for customers, and the secure, analytical administrative control panel for the store staff. These functional requirements were reviewed by Mr. Pradeep and broken down into structured, prioritized user stories in Jira, defining the roadmap for the eight-week redevelopment cycle.

2.3. Project 1: Complete Redevelopment of the LuckyMobile.lk Storefront

2.3.1. E-Commerce Usability Analysis and Requirements Elicitation

Before starting the development phase, the trainee worked under the guidance of Mr. Pradeep to conduct a usability analysis of the legacy LuckyMobile.lk web presence. The legacy system suffered from high latency during product catalogue rendering, layout inconsistencies when accessed on mobile screens, and a complicated checkout workflow that caused high cart abandonment rates. To resolve these challenges, the trainee gathered technical and functional requirements to plan a modern, high-performance solution.

The elicitation process focused on defining clear boundaries between two central applications: the Customer-Facing Storefront and the Administrative Control Panel. The functional specifications required a highly responsive customer storefront with dynamic filtering, instant search, and real-time inventory checks. For business administrators, the system needed secure login access, a centralized product inventory interface, and a validation-driven promotion scheduler. These requirements were translated into structured user stories in Jira, establishing a clear roadmap for the eight-week redevelopment cycle.

2.3.2. Relational Database Modeling and Prisma ORM Schema Design

With the system requirements defined, the trainee designed a relational database schema using PostgreSQL as the core storage engine. To simplify database operations and ensure type safety across the Node.js and Express.js applications, the trainee utilized Prisma Object-Relational Mapping (ORM). The database was modeled to support highly dynamic product specifications, multi-attribute variants, order processing, and administrative tracking.

A key design challenge was handling mobile phone variants, which require flexible combinations of attributes like storage capacity, color, and region. To maintain a clean, normalized relational model, the trainee structured the schema using related tables. Instead of storing variant combinations in a single table, the schema split them into a primary Product table connected to a 1-to-many ProductVariant table. These tables were supported by auxiliary tables for categories, brands, orders, and administrative users.

To document the database design for future maintenance, the schema details were organized into a clean structural definitions table.

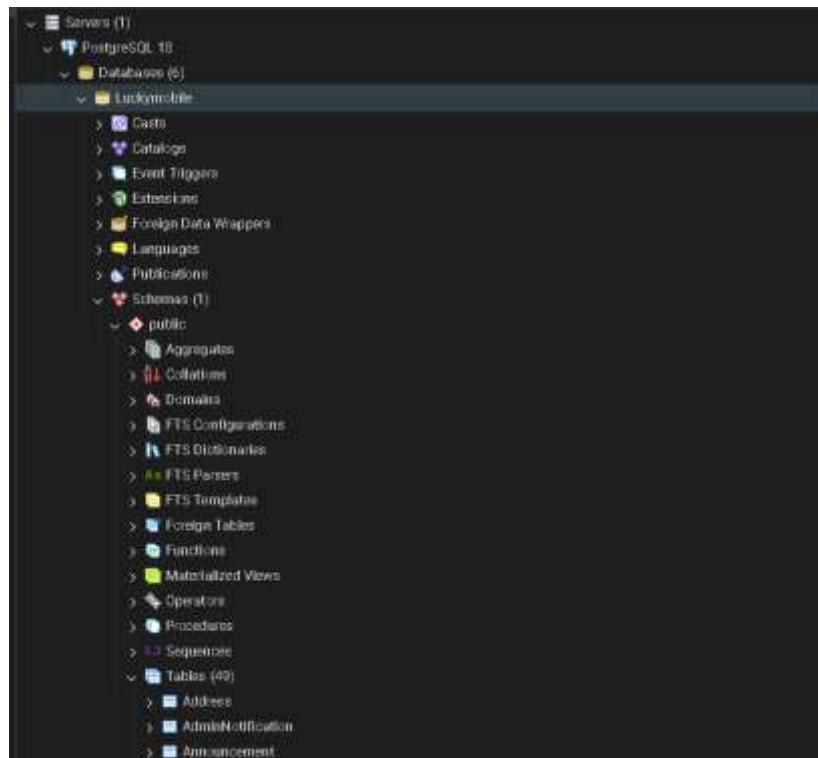


Figure 2.1 Prisma Configuration File

2.3.3. Layered Express.js Backend: Routes, Controllers, and Services

To build a maintainable backend application, the trainee organized the Express.js architecture into a clean, layered design. This structure separated core database operations, routing paths, and business logic into independent code modules, making the backend easier to read and maintain:

- **Routing Layer (/routes):** Handled incoming REST API requests, mapped the paths to their corresponding controller methods, and enforced input validation schemas using middleware.
- **Controller Layer (/controllers):** Acted as an entry point for requests, extracted query parameters, handled request-response lifecycles, and passed parameters to the service layer.
- **Service Layer (/services):** Contained the primary business logic of the application. Services executed database queries using Prisma Client, handled data transformations, and managed transactional operations.
- **Middleware Layer (/middlewares):** Executed shared tasks such as validating request payloads, catching system exceptions, and enforcing authorization rules.

By implementing this layered structure, the trainee ensured that database query logic was decoupled from routing paths, making the backend codebase easier to test, refactor, and scale.

2.3.4. Secure Administrative Authentication using JWT and bCrypt

Securing the administrative control panel was a critical security requirement. To prevent unauthorized access to sensitive inventory and sales dashboards, the trainee implemented a secure, stateless authorization module based on **JSON Web Tokens (JWT)** and **bCrypt password encryption**.

When an administrative user registers or updates their password, the plain-text string is processed with a salt round factor of 10 and hashed using bCrypt before being stored in the PostgreSQL database. This ensures that sensitive credentials are never stored in plain text. For user logins, the system compares the submitted password against the stored hash. Upon successful verification, the backend issues a signed, cryptographically secure JWT containing the user's ID and roles, which is sent back as an HTTP-only cookie. To protect admin routes, a custom validation middleware intercepts incoming requests, verifies the JWT signature, and checks the user's role before granting access.

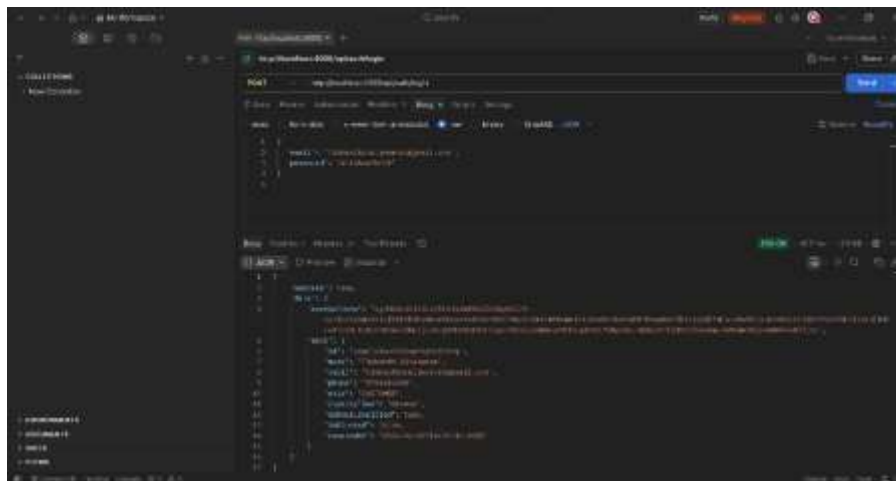


Figure 2.2 JWT Authentication Response

2.3.5. Frontend Application Architecture with Next.js and Tailwind CSS

With the backend APIs established, the trainee designed and built the customer storefront frontend using Next.js, React, TypeScript, and Tailwind CSS. The frontend architecture was

organized into a modular folder structure: **/components** for reusable UI elements like buttons, input fields, and product cards; **/layouts** for shared page structures; and **/pages** for independent views. This organization kept the codebase clean and maintainable.

To resolve the slow page rendering issues of the legacy website and deliver a fast shopping experience, the trainee implemented an advanced client-side state management and data caching pipeline. This setup combined TanStack Query (React Query) for managing server-state and Zustand as a lightweight client-state store:

- **TanStack Query Integration:** Used to manage all data-fetching operations from the Express APIs. By caching response data on the client side, the storefront can render product catalog grids instantly on subsequent page visits, running background requests to verify if any data has changed. This design significantly reduced API server load and eliminated loading screen delays for customers.
- **Zustand Global State Store:** Used to manage client-side state that does not require persistent server storage, such as shopping cart contents, active catalog search filters, and user session details. This lightweight store eliminated unnecessary React component re-renders, providing instant interface updates as users added items to their cart or adjusted filters.

Tailwind CSS was utilized to implement a utility-first styling model, allowing the trainee to build highly customized, responsive layouts that loaded instantly without the overhead of heavy, external CSS files, maintaining layout consistency across mobile devices and desktop monitors.

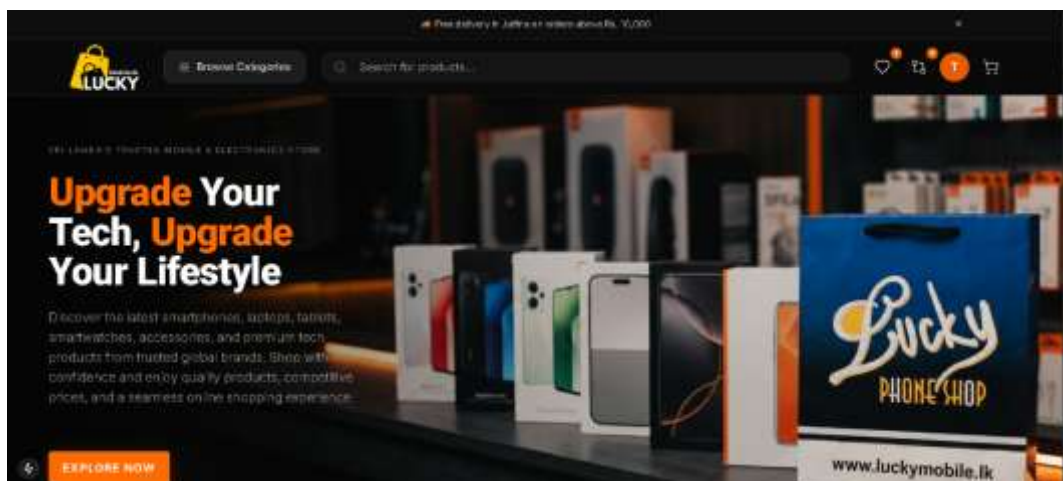


Figure 2.3 Storefront Homepage

2.3.6. Dynamic Variant Selection and WhatsApp Direct Ordering

One of the most complex customer-facing modules built during the redevelopment was the dynamic variant selector on the Product Detail Page (PDP). When purchasing a mobile phone, customers expect to dynamically select different configurations of storage capacity, color, and regional specifications.

To achieve this, the trainee designed a dynamic variant resolution algorithm on the frontend. When a user interacts with the selector buttons, the algorithm processes the selected attributes against the product's active variant array, dynamically updating the displayed price, stock status, and image galleries in real-time without requiring a page refresh. To prevent errors, the algorithm checks the database stock levels and immediately disables unavailable or out-of-stock attribute combinations.

To increase customer conversion rates and align the platform with local shopping preferences in Sri Lanka, the trainee designed and integrated a specialized WhatsApp Direct Ordering module on the Product Detail Page. In the domestic retail sector, many consumers prefer direct communication with merchants over executing standard, card-based checkout procedures.

When a customer clicks the green "Order via WhatsApp" button, a dedicated React handler extracts the currently selected product name, variant attributes (storage, color, region), and the dynamically resolved retail price. The handler compiles these variables into a pre-filled, structured message template and processes the string using URL percent-encoding.

The application then constructs a secure redirect URL utilizing the official WhatsApp Click-to-Chat API and triggers a safe browser redirect using standard security attributes to prevent potential tabnabbing vulnerabilities. This redirect opens the native WhatsApp application on mobile devices or WhatsApp Web on desktop browsers, allowing customers to instantly send their precise order requirements to the LuckyMobile sales team.

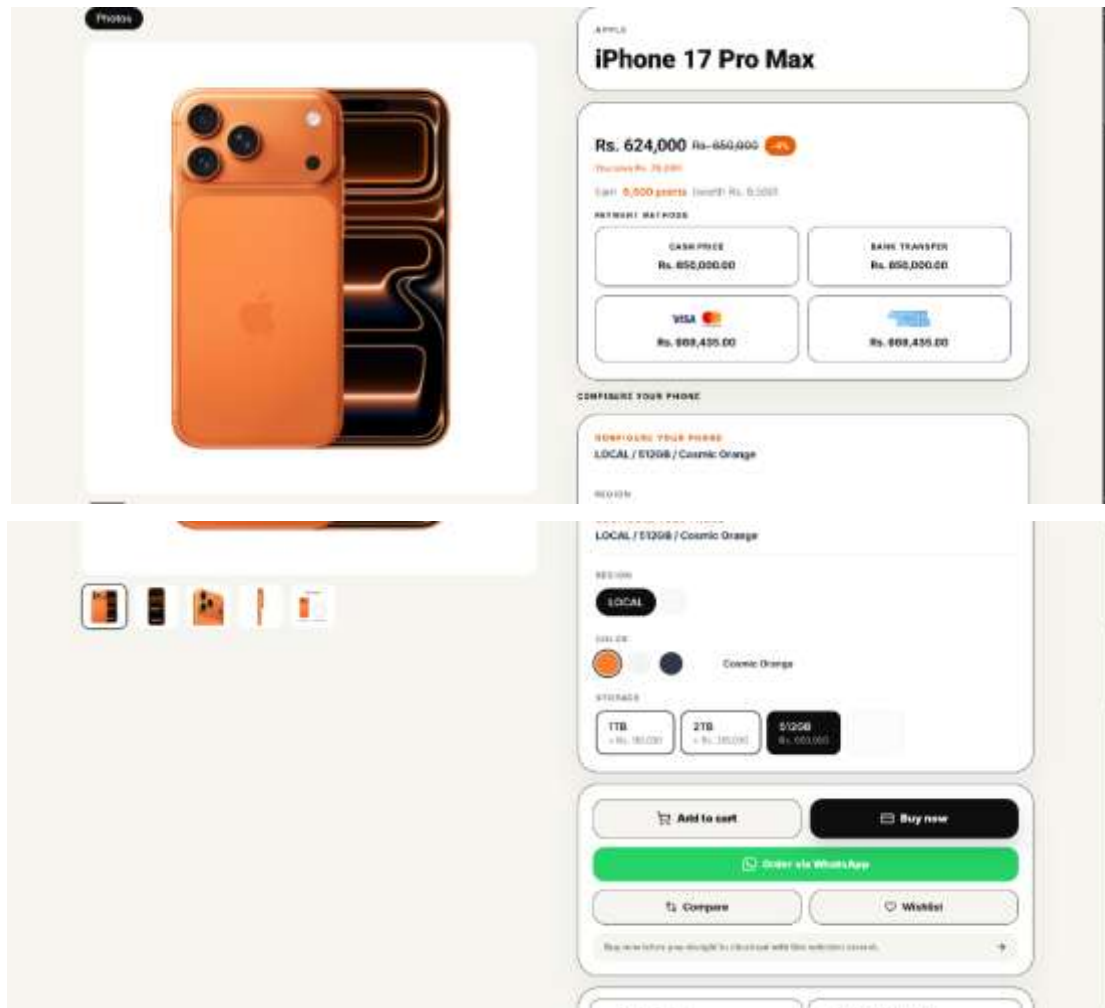


Figure 2.4 Product Detail Page

2.3.7. Overlap-Validated Promotion Scheduler and Admin Control Panel

The Administrative Control Panel was built to give business staff complete control over the LuckyMobile catalog, inventory levels, and promotional campaigns. The administration dashboard provided a clean, modern interface displaying sales analytics, low-stock alerts, active order logs, and product management tables.

A key administrative feature built by the trainee was the Promotion & Flash Sale Scheduler. This module allowed administrators to set promotional pricing for specific products over a defined timeframe. During testing, the trainee discovered that overlapping promotional schedules for the same product could cause pricing conflicts in the database. To prevent this, the trainee implemented a strict backend validation check within the creation service. Before saving a promotion, the service queries PostgreSQL to check for existing schedules for that

product ID that overlap with the new promotion's start and end times. If an overlap is detected, the transaction is blocked, and the admin panel displays a clear, informative validation error, ensuring complete data consistency.

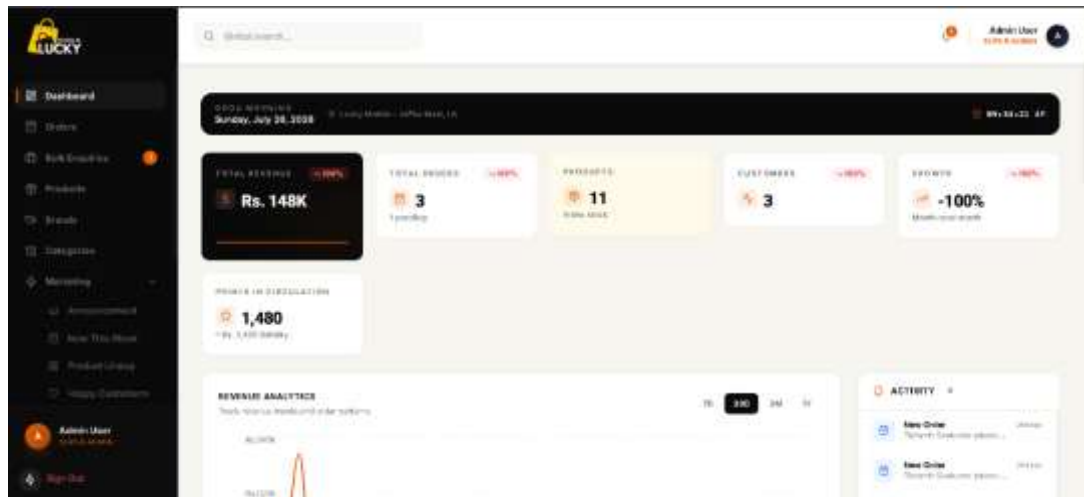


Figure 2.5 Admin Dashboard

2.3.8. Storefront Performance Optimization and Responsive Interface Validation

During the final phase of the redevelopment, the trainee shifted focus from feature implementation to performance optimization and quality assurance. To ensure the storefront loaded quickly on slow mobile networks, the trainee implemented performance optimization techniques:

- **Lazy Loading and Code Splitting:** Heavy UI components and off-screen images were loaded lazily, reducing the initial JavaScript bundle size and speeding up first-contentful-paint times.
- **Skeleton Screens:** Implemented animated skeleton placeholders across product grids to improve the perceived responsiveness of the application while waiting for API responses.
- **Responsive and Browser Testing:** Conducted extensive cross-device testing to ensure complete layout consistency and responsive scaling across Google Chrome, Safari, Firefox, and mobile web browsers.

By resolving minor layout bugs, optimising database queries, and reducing frontend bundle sizes, the trainee successfully delivered a polished, high-performance e-commerce platform that was ready for commercial deployment.

2.4. Project 2: MantraOne Multi-Tenant SaaS E-Commerce Platform

2.4.1. Software Architectural Principles of Shared-Database Multi-Tenancy

Following the successful completion of the LuckyMobile platform, the trainee was allocated to the design and implementation of MantraOne, an enterprise-grade Software-as-a-Service (SaaS) e-commerce platform designed to host multiple independent retail merchants within a single, unified codebase. During weeks 9 to 24, this project exposed the trainee to complex architectural paradigms and advanced system design. Unlike single-tenant applications that deploy separate code instances and database servers for each merchant, MantraOne leverages a shared-database, shared-application multi-tenant architecture. This pattern reduces cloud infrastructure costs, simplifies database maintenance, and allows platform-wide updates to be deployed to all merchants simultaneously.

In this architecture, multiple independent businesses—referred to as tenants—run on the same application instance and write to the same PostgreSQL database. The core engineering challenge is ensuring strict logical data isolation. A tenant must never be able to view, update, or corrupt another tenant's sensitive business data, customer profiles, or order history. To enforce these isolation boundaries end-to-end, MantraOne relies on a mandatory tenant identifier (**tenantId**) field embedded across all database entities. This identifier scopes every query executed by the database client, ensuring that database actions are restricted to the active merchant.

The platform enforces a three-tier user permission structure to maintain system security:

- **Super Admin:** The platform owner who manually provisions tenants and monitors global metrics via a secure platform panel.
- **Tenant / Shop Owner:** The merchant who accesses a dedicated admin panel to manage products, categories, orders, and customer accounts for their specific store.
- **Customer:** The end-shopper who browses and purchases items from the merchant's public storefront.

The tenant identifier is always resolved securely on the backend from JSON Web Tokens or routing middleware, and is never accepted from the request body, preventing malicious actors from manipulating parameters to access unauthorized merchant data.

2.4.2. Centralized Storefront Router and Tenant Resolution Middleware

To route incoming traffic to the correct business context, the trainee developed a centralized storefront router and tenant resolution middleware within the Express.js backend. Because customers access independent storefronts powered by the same running backend, the platform must dynamically resolve the tenant's identity before executing any product catalog, cart, or checkout logic. This resolution is performed on every storefront request using a customized tenant resolution middleware.

When an API call is dispatched from the customer storefront, the middleware intercepts the incoming request and extracts the shop's identifier. This is done by reading a custom HTTP request header named **x-shop-slug**. If the header is missing, the middleware fallback logic parses the request's host header to extract the storefront's subdomain. Once the slug is obtained, the middleware queries the PostgreSQL database via Prisma ORM to find the matching tenant record.

If no tenant matches the slug, the middleware terminates the request immediately and returns a standardized HTTP 404 client error. If the tenant is found but its status is flagged as suspended or archived, the middleware returns an HTTP 403 forbidden response, blocking further execution. Once the active tenant is successfully validated, the middleware binds the resolved tenant identifier directly to the Express request object as a session property and invokes the next function in the middleware chain.

By binding the tenant identifier at the entry point of the API, all downstream controllers and database services can access the identifier directly from the request object, eliminating duplicate routing code and ensuring that all data operations are scoped securely to the active merchant.



Figure 2.6 Authentication API Endpoints

2.4.3. Scoped Database Migrations and Prisma Transaction Integrity

The database layer of the MantraOne monorepo resides within a shared database package, isolating the Prisma schema and database configuration from individual applications. To manage schema changes in a multi-tenant environment, the trainee utilized Prisma Migrate to define and run incremental database updates. Pushing schema migrations to a shared database requires careful planning, as any destructive schema modification can disrupt operations for all active storefronts on the platform.

To prevent database corruption and maintain data consistency, the trainee utilized Prisma's programmatic transaction API (**\$transaction**) when implementing complex data creation services. Multi-tenant database entries often require writing records across multiple related tables simultaneously. For example, when an administrative user creates a new retail product, the system must write a primary row to the **Product** table and multiple associated rows to the **ProductVariant** table to store color, size, and pricing options.

If a backend failure or database connection drop occurs after creating the primary product row but before inserting the variant records, the database is left with a orphaned product record. This partial write corrupts the product catalog and causes runtime errors on the customer storefront. To eliminate this risk, the trainee structured the product creation services inside an atomic database transaction.

This transactional wrapper ensures that all database operations within the block succeed or fail together. If any query within the transaction fails, the database engine automatically rolls back all modifications made during the session, restoring the database to its pre-transaction state and preserving data integrity across all merchant accounts.

Table 2.1 Multi-Tenant Database Isolation Strategy

Database Collection	Isolation Mechanism	Index Optimization	Data Access Scope
Tenant	Primary Account Key	Unique Index on subdomain	Global Cloud Routing Table
Product	Composite Key (id, tenantId)	Composite Index on (tenantId, categoryId)	Storefront Inventory Catalog
Order	Foreign Key (tenantId)	B-Tree Index on tenantId	Tenant-Specific Order Records

Database Collection	Isolation Mechanism	Index Optimization	Data Access Scope
Customer	Dynamic Relationship Table	Composite Index on (tenantId, email)	Tenant-Isolated Customer Management

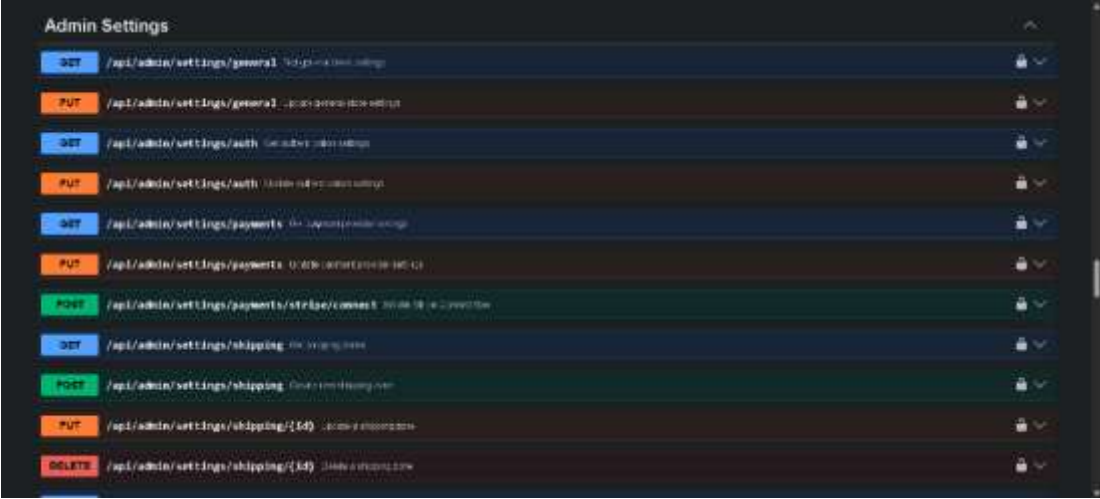
2.4.4. Storefront Branding Engine and CSS Variables Theme Resolution

To allow each merchant to customize the visual appearance of their storefront without modifying the core codebase, the trainee implemented a dynamic, configuration-driven branding engine. Through the centralized setting module in the admin panel, shop owners can configure their business logos, promotional banners, custom fonts, and core branding colors. The storefront application dynamically fetches these branding configurations when a customer loads the storefront.

To resolve themes dynamically while sharing a single Next.js frontend application, the trainee designed a layout system based on CSS Custom Properties (CSS variables). When the storefront application resolves the active tenant via the custom request headers, it dispatches an API request to the settings endpoint to retrieve the tenant's active design tokens.

The application then injects these tokens as dynamic CSS variables directly into the document root of the rendered page. Standard utility-first Tailwind classes across all reusable storefront components reference these dynamic variables instead of utilizing hardcoded color values. Under this model, changing a primary brand color in the admin panel updates the injected inline style variables, causing the storefront buttons, highlights, and borders to update their appearance instantly.

This styling separation allows independent storefronts to render completely distinct visual identities—such as a dark-toned luxury interface for one merchant and a bright, vibrant layout for another—while sharing the same Next.js codebase, demonstrating the flexibility of the platform's multi-tenant architecture.



The screenshot shows a table titled 'Admin Settings' with columns for HTTP method, endpoint, and description. The methods are color-coded: GET (blue), PUT (orange), POST (green), and DELETE (red). Each row also includes a lock icon and a dropdown arrow.

Method	Endpoint	Description
GET	/api/admin/settings/general	Get general admin settings
PUT	/api/admin/settings/general	Update general admin settings
GET	/api/admin/settings/auth	Get auth admin settings
PUT	/api/admin/settings/auth	Update auth admin settings
GET	/api/admin/settings/payments	Get payment admin settings
PUT	/api/admin/settings/payments	Update payment admin settings
POST	/api/admin/settings/payments/storage/connect	Connect to storage provider
GET	/api/admin/settings/shipping	Get shipping admin settings
POST	/api/admin/settings/shipping	Create new shipping zone
PUT	/api/admin/settings/shipping/{id}	Update shipping zone
DELETE	/api/admin/settings/shipping/{id}	Delete shipping zone

Figure 2.7 Settings API Endpoints

2.4.5. Integrating AWS S3 for Private Multi-Tenant Media Storage

In a multi-tenant SaaS application, hosting and separating media assets is as critical as isolating database tables. To replace unscalable local storage options, the trainee integrated Amazon Web Services (AWS) Simple Storage Service (S3) to host and isolate product images and marketing assets. The integration allows administrative users to upload high-resolution media through the admin panel.

When an administrator submits an image, the file is intercepted on the backend by a multi-part upload middleware. The middleware validates the file size to prevent large files from wasting server storage and checks the MIME type to block malicious file uploads. Verified files are then streamed directly to a secure AWS S3 bucket.

To comply with platform development guidelines, the backend uses the runtime's native JavaScript fetch API rather than external HTTP client libraries to handle S3 uploads, reducing external dependencies and improving upload speeds. To maintain strict tenant isolation within the storage bucket, the system structures upload paths using tenant-scoped directory prefixes.

Images are saved in folders organized by the tenant's unique identifier and target module, such as `uploads/{tenantId}/products/{image_uuid}.webp`. This path structure prevents filename collisions between different merchants, simplifies storage usage tracking, and ensures that merchants can only access and modify their own media assets.

Figure 2.8 Product Creation Form

2.4.6. End-to-End Order Processing and Atomic Inventory Synchronization

The order management pipeline is a core component of the platform, linking products, customer profiles, payment settings, and real-time inventory systems. The trainee developed backend APIs to manage the complete customer purchase lifecycle from initial checkout validation through to administrative order fulfillment.

When a customer submits a new checkout payload, the backend initiates a series of validation checks within a secure database transaction block. First, the system queries the ProductVariant and Inventory tables to verify that each item in the customer's cart is in stock. To prevent race conditions where multiple users attempt to purchase the same low-stock item simultaneously, the transaction locks the corresponding variant rows in the database.

After confirming stock availability, the database service decrements the stock counts, writes a new record to the Order table with a status of pending, and links the order to the active tenant's unique identifier. This atomic sequence ensures that inventory counts are synchronized with finalized orders under high traffic, preventing errors such as overselling. Once the transaction completes, the system updates the frontend checkout interface, confirming the order and allowing the merchant to process the shipment.

2.4.7. Security Audit: Direct URL Access Bypass and Custom Route Guards

During week 16 of the industrial training placement, the trainee performed a comprehensive security audit of the MantraOne Admin Panel. While the interface was configured to hide restricted navigation links based on user roles (e.g., hiding financial and settings tabs from

standard sales accounts), the trainee identified a critical security vulnerability: users could bypass these interface restrictions by typing protected page URLs directly into the browser's address bar.

To resolve this vulnerability, the trainee implemented a multi-tiered security pattern utilizing Next.js custom route guards on the client side, and role-based middleware on the backend. Client-side route guards are managed in the layout wrapper of the dashboard application. When a user attempts to open a page, the route guard intercepts the transition, decodes the user's role from their authentication token, and checks their permissions against the page requirements.

If the user is unprivileged, the guard blocks the page render and redirects them to a secure access-denied page. Because client-side checks can be bypassed by modifying local state, the trainee implemented corresponding authorization middleware on the Express.js backend. This middleware validates the user's role and scopes on every incoming REST API request under the `/api/admin/` prefix, ensuring complete, system-wide protection.



Figure 2.9 Customer Management Module

2.4.8. Event-Driven Real-Time Order Alerts using Socket.IO

To help merchants monitor customer activity and fulfill purchases quickly, the trainee implemented a real-time notification engine within the admin panel using Socket.IO. Standard REST APIs require the admin panel to periodically poll the backend to check for new orders, which introduces latencies and creates unnecessary query load on the PostgreSQL database. Bidirectional WebSockets solve this by establishing persistent, low-latency connections between clients and the server.

To support WebSockets, the trainee configured the Express.js server to run on an HTTP server instance rather than calling Express listen directly, ensuring that the WebSocket protocol handshake is handled correctly. When a customer completes a purchase on a storefront, the order controller triggers an order-created event.

The Socket.IO server intercepts this event, extracts the associated tenant identifier, and emits a notification payload exclusively to clients connected to that merchant’s admin panel session. This event-driven design ensures that administrators receive instant audio-visual alerts the moment an order is placed, allowing them to initiate packaging and delivery processes immediately without having to refresh their browsers.



Figure 2.10 Notification API Endpoints

2.4.9. Automating Transactional Emails with Scoped Nodemailer Services

To automate customer communication workflows, the trainee configured a transactional email service utilizing Nodemailer and secure cloud SMTP relays. When customers complete important events—such as registering a new account, placing an order, or requesting a password reset—the system generates and sends automated email notifications.

The email service was built using a reusable, modular design. The trainee created structured, responsive HTML email templates for each event type, using dynamic placeholders to inject transaction details (such as customer names, order lists, verification tokens, and prices).

When an event occurs, the corresponding service class compiles the template, initializes an SMTP transport channel, and sends the email. To ensure high deliverability and protect the sender reputation, the email operations run as background tasks, preventing network delays from affecting main application performance.

2.4.10. Storefront Search Optimization and State-Preserving Product Filters

During the final phase of the project, the trainee worked to optimize the storefront's search and product discovery experience. To help customers find products quickly in large catalogues, the trainee built optimized backend search services capable of processing multiple filters simultaneously, including categories, brands, price ranges, and stock availability.

To ensure a smooth user experience, the trainee implemented state-preserving URL filters on the frontend. When a user applies dynamic filters or types search terms, the application serializes these selections directly into the browser's URL query parameters.

This design allows customers to copy and share the URL with their active filters preserved, and ensures that their search settings remain intact even if they refresh the webpage. The backend search queries were optimized using indexing and database projections, minimizing query times and ensuring the storefront remains fast and responsive during high-volume customer browsing.

2.5. Industry Processes, Office Practices, and Technical Tooling Specifications

2.5.1. Professional Production Processes and Quality Assurance

To bridge the gap between academic theory and corporate standards, the trainee participated in the standard software development lifecycle (SDLC) followed at Matrix Mantra. All engineering projects are managed under a strict Quality Assurance (QA) and Continuous Integration (CI) framework. When assigned a feature in Jira, the trainee followed a rigorous, multi-tiered verification workflow:

1. **Local API Verification:** The trainee tested all newly developed REST endpoints in isolation using Postman collections, verifying that the controller layer returned correct JSON structures and standard HTTP error status codes.
2. **Linting & Pre-Commit Gates:** Code changes were ran through automated code formatters and static analyzers (ESLint, Prettier) to ensure compliance with corporate style guides before being pushed to GitHub.
3. **Git-flow Branching Strategy:** Code modifications were developed on isolated feature branches created off the centralized develop branch.
4. **Pull Request (PR) Inspections:** Completed features were submitted as PRs, triggering

automated build checks via GitHub Actions.

5. **Manual Peer Code Review:** Each PR was manually reviewed by Technical Lead Mr. Pradeep, who analyzed execution path efficiency, database transaction safety, and role-based middleware security before approving the code merge into the development branch.

This production pipeline taught the trainee the importance of writing secure, high-quality, and highly maintainable code.

2.5.2. Administrative, Office, and Collaborative Workspace Practices

Matrix Mantra's remote-first and hybrid operational model required a highly organized administrative and collaborative workflow to keep distributed development teams aligned. The trainee actively participated in all standard Agile Scrum ceremonies to maintain coordination with cross-functional team members, including product owners, software engineers, and QA specialists:

- **Sprint Planning:** Bi-weekly sessions to estimate task complexities using story points, prioritize deliverables, and commit to sprint scopes within Jira.
- **Daily Scrum Standups:** Structured morning syncs conducted via Slack or Zoom where each developer shared their progress, outlined daily tasks, and raised technical blockers.
- **Sprint Reviews:** Iterative demonstration sessions where completed feature modules were presented to business stakeholders for feedback before public release.
- **Sprint Retrospectives:** End-of-cycle reviews to reflect on development bottlenecks, suggest system improvements, and continuously optimize team workflows.

Additionally, the trainee completed standard administrative onboarding processes, which included studying internal coding standard documentations, signing standard Non-Disclosure Agreements (NDAs), and receiving security briefings on standard data protection regulations.

2.5.3. Financial Procedures and Commercial Client Operations

During the development of the e-commerce systems, the trainee was exposed to the financial workflows and commercial considerations that drive contemporary digital platforms. This exposure was particularly valuable during the architectural design of the MantraOne multi-

tenant platform. While standard single-tenant sites focus on direct product sales, SaaS environments require managing complex, automated billing systems and service-level commitments.

The trainee studied and modeled several business-centric workflows:

- **Service-Level Agreement (SLA) Calculations:** Analyzing response time and uptime requirements to design reliable, low-latency database queries.
- **Transaction Processing:** Understanding cash-on-delivery placeholder flows and designing order state models to support future integration with payment gateways like Stripe or PayPal.
- **Subscription & Service Fee Models:** Reviewing the SaaS monetization roadmap to plan the database structures required for automated tenant subscription tracking, billing cycles, and platform usage analytics.

This commercial context helped the trainee understand how technical decisions—such as query performance, server state caching, and database transaction safety—directly impact a business's operational costs and client satisfaction levels.

2.5.4. Technical Equipment, Developer Tooling, and Environmental Specifications

To deliver commercial-grade software products, developers must utilize specialized equipment, runtime environments, and integrated development tools. Throughout the internship, the trainee operated within a highly optimized technical environment:

- **Primary Hardware Specifications:** DELL Latitude 3490 Laptop, equipped with an Intel Core i5 8th Gen Processor, 16 GB of DDR4 RAM, and a 64-bit Windows 11 Operating System, paired with A4Tech HS-30i on-ear headphones.
- **Development Runtime & Packages:** Node.js v20.x runtime, running Express.js for backend APIs, Next.js v14 (App Router) for frontend storefronts, Prisma Client for database operations, and TypeScript to enforce complete static type safety.
- **Database Engine & Tools:** PostgreSQL v16 relational database server, managed locally via DBeaver, and configured with automated Prisma schema migrations.
- **Version Control & CI/CD:** Git versioning managed through GitHub repositories, using GitHub Actions to coordinate linting rules, formatting checks, and compilation

tests.

- **API Exploration & Interaction:** Swagger UI (OpenAPI 3.0 specs) and Postman collections to explore, test, and document REST API routes.
- **Integrated Development Environment (IDE):** VS Code configured with ESLint, Prettier, and Prisma extensions to maintain consistent code formatting and simplify backend development.

2.5.5. Occupational Health and Safety (OHS) and Office Safety Procedures

During the twenty-four-week industrial placement, the trainee was personally exposed to and required to adhere to standard Occupational Health and Safety (OHS) and safety procedures optimized for modern software engineering environments. In a hybrid and remote development setup, these safety procedures focus heavily on workstation ergonomics and preventative physical health to mitigate the risks associated with prolonged screen time and sedentary work.

The trainee was trained to configure their local workspace in compliance with ergonomic safety guidelines. This included utilizing adjustable ergonomic seating with proper lumbar support, positioning monitor displays at eye level to prevent neck strain, and organizing input hardware to support natural wrist alignment, reducing the risk of Repetitive Strain Injuries (RSI) like Carpal Tunnel Syndrome.

To prevent optical fatigue, eye strain, and physical exhaustion during intense full-stack development and database migrations, the trainee followed company safety guidelines. These recommended scheduled physical stretches and the regular practice of the 20-20-20 rule (resting the eyes by looking at an object 20 feet away for 20 seconds every 20 minutes).

Additionally, during on-site collaborative visits to the Jaffna development center, the trainee was oriented on physical office safety procedures. This orientation covered clean pathway maintenance, proper under-desk cable management to eliminate electrical and tripping hazards, and standard building emergency and fire evacuation protocols. Adhering to these structured safety procedures taught the trainee how to maintain physical well-being and operational safety within a demanding, high-performance IT workplace.

2.6. Reflective Analysis of Technical Hurdles, Personal Gains, and Weaknesses

2.6.1. Technical Challenges, Work-Related Difficulties, and Implemented Solutions

During the twenty-four-week placement, the trainee encountered several complex technical challenges. Overcoming these difficulties was central to their professional growth as an independent software engineer:

- **Prisma Relationship Configurations & Migration Failures:** During the initial database design of the LuckyMobile platform, the trainee faced migration failures due to incorrectly modeled multi-table relationships and circular foreign-key dependencies. To solve this, the trainee studied database normalization principles, restructured the Prisma schema into related Product and ProductVariant tables, and ran migrations successfully.
- **Dynamic Variant Sync Lag on Storefront:** Rapid user interactions with the variant selector on the product detail page initially caused mismatched price rendering due to asynchronous state lag in React. The trainee resolved this by simplifying the React state update sequence, ensuring all downstream calculations used the latest selected variant reference before rendering the UI.
- **Admin Panel Direct URL Access Bypass:** During the security audit of the MantraOne admin panel, the trainee discovered that restricted pages could be accessed directly by typing the URL into the browser, bypassing client-side navigation menus. The trainee resolved this vulnerability by implementing Next.js custom route guards on the frontend and Express backend authorization middleware to validate user permissions for every API request.
- **Multi-Tenant Cache Leak and State Carryover:** During early testing of the MantraOne storefront, switching between tenant domains occasionally displayed elements of the previous store's cached logo or colors. The trainee resolved this state carryover issue by updating the Zustand state manager to clear all cached values and force-rehydrate theme configurations when switching domains.

2.6.2. Communication and Coordination Difficulties in Hybrid Workspaces

In addition to technical hurdles, the trainee faced operational difficulties associated with the remote-first and hybrid software engineering model. As this was the trainee's first professional industry experience, adjusting to remote team coordination posed initial challenges:

- **Remote Work Isolation:** Working from home made it difficult to build initial rapport

with team members and understand the company's operational dynamics compared to a physical office environment.

- **Technical Blocker Synchronization:** Resolving technical blockers remotely was slower than on-site pairing, as describing complex codebase issues via text or screenshares required precise articulation.
- **Global Stakeholder Alignment:** Managing sprint priorities for global commercial contracts required the team to remain highly adaptable, adjusting task allocations to meet changing deadlines.

The trainee successfully addressed these challenges by participating actively in all virtual daily scrums, communicating technical questions clearly on Slack, documenting their coding decisions, and attending the physical Matrix Mantra Company Meetup & Team Gathering in Week 18 to build strong, face-to-face professional relationships with senior developers and mentors.

2.6.3. Evaluation of Professional and Interpersonal Gains

The industrial placement proved to be highly valuable, allowing the trainee to develop comprehensive technical capabilities and essential soft skills.

The core professional and interpersonal gains achieved include:

- **Advanced Full-Stack Engineering Expertise:** Transitioning from basic programming to designing enterprise-grade software architectures, mastering Next.js rendering paths, Express service layers, and Prisma transactions.
- **Database Modeling & Security Proficiency:** Gaining a strong understanding of database normalization, relational schema design, JWT authentication flows, and multi-tenant data isolation.
- **Agile Project Management Experience:** Mastering professional software tools (Jira, Git, monorepos) and participating in standard Agile scrum ceremonies.
- **Professional Confidence & Communication:** Overcoming initial nervousness to participate actively in team discussions, present technical sprint demonstrations, and collaborate with cross-functional professionals.
- **Problem-Solving Independence:** Developing a structured approach to debugging, identifying the root causes of issues, and researching official documentation to

implement reliable solutions.

2.6.4. Honest Identification of Personal Weaknesses and Areas for Growth

While the internship was highly successful, it also helped the trainee identify personal weaknesses and areas that require continuous improvement:

- **Initial Speaking Nervousness in Multilingual Environments:** At the start of the internship, the trainee felt hesitant to speak English during formal sync-ups and collaborative sprint planning meetings. Engaging in regular discussions and receiving supportive feedback from Mr. Pradeep helped the trainee build professional confidence.
- **Git-flow Adaptability & Pull Request Revisions:** In the early weeks, several of the trainee's Pull Requests required multiple revisions to align with professional code standards, as the trainee was still adjusting to enterprise-level branching, linting rules, and formatting guidelines. The trainee resolved this by creating a personal checklist for self-review before submitting code.
- **Time Management & Single-Issue Over-Allocation:** During complex debugging tasks, the trainee occasionally spent excessive time trying to resolve issues independently before seeking guidance, which temporarily impacted sprint velocity. Over time, the trainee learned to balance self-research with proactive team communication, raising technical blocks during daily scrums.

2.6.5. Perspective on Skills and Experiences Not Gained During the Placement

A complete, objective internship review requires identifying the technical skills and professional experiences that were not obtained due to the nature and constraints of the assigned projects:

- **Lack of Production Deployment Feedback:** Because the MantraOne SaaS platform was an ongoing, internal project in an active development phase (reaching approximately 60% completion during the internship), the trainee did not experience deploying a SaaS platform to a live production environment or handling real user onboarding and adoption feedback.
- **Absence of Payment Gateway Integration:** The e-commerce projects utilized Cash on Delivery placeholders, leaving key payment gateway integrations (such as Stripe or PayPal SDK configurations) on the platform's future product roadmap.

- **No Exposure to Containerized Deployment (Docker):** The trainee configured their development environment locally, lacking hands-on exposure to containerizing applications with Docker or setting up cloud orchestration pipelines.
- **Limited Direct Client Interaction:** Because both redevelopments were managed internally, the trainee did not have opportunities to engage directly with client business owners or end-users during requirements elicitation, relying instead on business analysts to represent user perspectives.

CHAPTER 3

CONCLUSION AND RECOMMENDATIONS

3.1. Synthesis and Summary of the Industrial Training Experience

The twenty-four-week industrial training placement completed at Matrix Mantra (Pvt) Ltd was a highly transformative professional milestone that successfully bridged the gap between undergraduate academic theory and commercial software engineering. Spanning from February 2, 2026, to July 31, 2026, the internship exposed the trainee to a comprehensive range of full-stack development methodologies, relational database design principles, and modern collaboration workflows.

The training was structured into two core phases. The initial phase focused on the complete re-engineering of the LuckyMobile.lk electronics and mobile retail storefront, where the trainee utilized Next.js, React, Node.js, Prisma ORM, and PostgreSQL to resolve performance bottlenecks and implement a polished, direct-to-messenger WhatsApp ordering module.

The second phase involved contributing to the core development of the MantraOne multi-tenant Software-as-a-Service (SaaS) platform. This work provided deep technical exposure to complex architectural patterns, including centralized tenant resolution middleware, atomic database transactions, CSS custom property dynamic branding, and secure role-based API guards.

Beyond developing specific technical skills, the placement allowed the trainee to gain a thorough understanding of the software development life cycle under the Agile Scrum framework, improving key professional skills such as requirements analysis, task estimation,

peer code reviews, and remote team communication.

3.2. Critique of the Training Establishment (Matrix Mantra)

3.2.1. Evaluation of Mentorship, Training Personnel, and Work Environment

Matrix Mantra (Pvt) Ltd demonstrated exceptional capability as a training establishment, providing a highly productive and modern environment that fosters rapid technical development, independence, and professional growth. Despite operating on a hybrid model, the company has built an organized virtual workspace that replicates the collaboration and accountability of an on-site office.

The organizational culture at Matrix Mantra is highly supportive of learning. Trainees are treated as active software engineers rather than passive observers, receiving direct exposure to real-world commercial requirements and enterprise-grade codebases.

The effectiveness and enthusiasm of the training personnel were outstanding throughout the internship. Under the leadership of Managing Representative Mr. Mithun Paresith and the direct supervision of Mr. Pradeep, who acted as Project Manager and Technical Lead, the trainee received continuous, structured mentorship.

Mr. Pradeep maintained a genuine interest in guiding the trainee, dedicating time to daily scrums, architectural discussions, and detailed code reviews. Rather than merely assigning tasks, the mentorship focused on explaining the fundamental engineering principles behind database normalization, data security, and performance optimization, which significantly accelerated the trainee's growth as an independent software engineer.

3.2.2. Identified Deficiencies and Suggestions for Organizational Improvement

While Matrix Mantra excels at providing high-quality, hands-on software development training, a few operational deficiencies were identified during the internship:

- **Steep Learning Curve and Documentation Gaps:** Due to the complexity of the platform architectures—particularly the unified Git monorepo, Prisma relational database layer, and tenant isolation middleware—onboarding junior developers and

trainees can be challenging. The company currently lacks comprehensive, step-by-step internal setup guides and documentation for these systems, resulting in an unnecessarily long onboarding transition. It is suggested that the company establish a centralized, up-to-date developer wiki detailing local environment configurations, branching strategies, and testing guidelines to streamline future trainee onboarding.

- **Centralization of Key Supervisory Roles:** Technical leadership and final code quality validation are heavily concentrated within a few key roles. Mr. Pradeep personally conducts all code reviews and architectural audits, which can create a bottleneck during peak development sprints. The company would benefit from introducing intermediate technical lead positions, allowing senior developers to share the review load and ensuring a smoother, more scalable development pipeline.
- **Friction in Remote Coordination:** While remote work offers flexibility, it can occasionally lead to communication friction and delayed feedback on pull requests compared to a physical office environment. Matrix Mantra could improve collaboration by setting up structured, bi-weekly virtual pair-programming sessions and quick daily sync-ups, ensuring that technical blockers are resolved more efficiently.

3.3. Critique of the University and NAITA Industrial Training Programme

3.3.1. Evaluation of University Academic Curriculum and Relevancy

An essential objective of the industrial training module is evaluating the relevance and completeness of the B.Sc. (Hons) in Information Technology curriculum offered by the Faculty of Information Technology, University of Moratuwa, against the active engineering demands of the software industry. During the twenty-four-week placement, the trainee actively applied concepts from several academic modules to solve real-world problems.

The core database concepts taught in university modules laid a strong foundation for designing normalized schemas in PostgreSQL, while academic web technology modules provided the essential principles of HTTP requests, client-server models, and styling. Additionally, the software engineering modules proved highly relevant, as the Scrum workflows, sprint planning, and task estimation procedures utilized daily at Matrix Mantra directly mirrored academic Agile frameworks.

However, the rapid evolution of modern software development has highlighted key areas where the academic curriculum could be updated to better align with contemporary industry standards:

- **Introduction of Modern Full-Stack Frameworks:** While the academic modules provide an excellent grasp of raw HTML, CSS, JavaScript, and basic SQL queries, modern enterprise development relies heavily on advanced framework abstractions. Undergraduates would benefit immensely from the introduction of elective workshops or modules focusing on component-driven frontend frameworks (such as React and Next.js) and type-safe backend engines (using TypeScript and Prisma ORM).
- **Version Control and Distributed Workflows:** In a professional software agency, version control is not merely a tool but a core collaboration pipeline. The trainee experienced initial difficulties adapting to complex Git-flow branching, pull requests, and merge conflict resolutions. Integrating mandatory Git training and team-based monorepo projects into early-level academic coursework would significantly reduce transition friction for future interns.
- **Containerization and Cloud Services:** Modern software product engineering is tightly coupled with cloud infrastructures (such as AWS S3 storage, CDN distributions, and containerized deployments). Introducing basic concepts of Docker and cloud-based asset handling into the academic curriculum would equip undergraduates with the deployment competencies highly valued by global technology firms.

3.3.2. Evaluation of NAITA Coordinated Administration and Monitoring Processes

The National Apprentice and Industrial Training Authority (NAITA) plays a crucial role in regulating, administering, and coordinating the industrial placement of undergraduates, ensuring that the training environment meets national vocational and professional standards. The administrative coordination, beginning with the registration of apprentices and the formal issuance of the daily training logbooks, provides a structured framework that instills workplace discipline, professional accountability, and consistent progress tracking.

However, based on the practical observations made during this twenty-four-week hybrid internship, several comments and constructive suggestions can be made to modernize and improve the NAITA-coordinated aspects of the programme:

- **Transition from Physical Logbooks to Digital Portals:** NAITA currently relies on a

manual, physical daily diary and paper-based monthly report sheets. This physical record-keeping process is highly repetitive, time-consuming, and prone to physical damage or loss. Since contemporary software development firms operate almost entirely in digitized, virtual environments, NAITA should collaborate with the University of Moratuwa to establish a secure, centralized digital diary portal. A digital system would allow trainees to log daily tasks electronically, enable industry supervisors to sign off on progress with digital signatures, and provide NAITA officers with real-time access to progress records, drastically reducing manual administrative overhead.

- **Industry-Aligned Performance Monitoring and Assessment:** The supervisory monitoring visits conducted by NAITA representatives are highly valuable for ensuring trainee welfare and workplace compliance. However, these evaluations often place a heavy emphasis on checking physical signature counts and logging hours rather than assessing the actual technical contributions of the student. To better align with the software industry, NAITA's assessment criteria should be updated to evaluate project-based deliverables, such as the trainee's contribution to software architectures, sprint velocities in Jira, and adherence to clean code guidelines during peer reviews.
- **Pre-Placement briefings on NAITA Coordination:** The administrative registration and logbook handovers before commencing training can occasionally be confusing for undergraduates. NAITA could improve the onboarding experience by conducting a structured, pre-placement briefing session in coordination with the university's training engineer. Providing a detailed, digital handbook outlining registration steps, logbook maintenance rules, and the criteria for successful continuous evaluations would eliminate initial confusion and ensure a smoother transition into the corporate sector.

3.4. Expectations Met, Personal Growth, and Professional Gains

The industrial placement successfully met and exceeded the trainee's personal and professional expectations. Entering the internship, the primary goal was to gain practical full-stack coding experience and understand how commercial web systems are built. Through active participation in the LuckyMobile storefront redevelopment and the MantraOne SaaS platform, the trainee not only achieved this goal but also developed a strong appreciation for database transaction safety, software design patterns, and application security. The experience shifted

the trainee's perspective from merely writing functional code to engineering scalable, maintainable, and secure software systems.

In addition to technical growth, the internship was a major driver of personal and interpersonal development. Operating within a hybrid and remote team environment helped the trainee overcome initial nervousness and build professional confidence.

Participating in daily scrums, communicating technical blocks clearly on Slack, and conducting feature demonstrations during team syncs significantly improved the trainee's verbal communication, time management, and professional accountability.

Furthermore, participating in the physical Matrix Mantra Company Meetup & Team Gathering during Week 18 allowed the trainee to interact directly with senior developers and managers, fostering team trust and highlighting the value of a collaborative workspace culture.

3.5. Actionable Recommendations for Future Trainees

Based on the technical, professional, and personal experiences gained during the placement, the following recommendations are offered for future University of Moratuwa undergraduates to maximize their industrial training learning:

- **Master Git and Distributed Workflows Prior to Placement:** Before starting an internship, allocate time to master command-line Git utilities, branching strategies, commit guidelines, and pull request mechanics. Professional workspaces operate on shared code bases, and having a solid grasp of version control prevents early-stage merge conflicts and development bottlenecks.
- **Prioritize Architectural Understanding Over Fast Coding:** When assigned a development task in a professional repository, resist the urge to copy-paste solutions or rush through the implementation. Spend time studying the existing directory structure, understanding how data flows between the architectural layers, and identifying the system's design rules.
- **Proactively Communicate Technical Blockers:** Do not hesitate to seek guidance from senior developers when a technical blocker cannot be resolved after reasonable self-research. When raising a blocker during daily scrums, clearly articulate what has been attempted, what failed, and what resources were referenced, as this demonstrates a

proactive, problem-solving attitude.

- **Embrace Feedback and Code Reviews:** Treat code reviews not as personal criticisms but as highly valuable learning opportunities. Pay close attention to suggestions made by technical leads regarding transaction safety, variable naming conventions, and query optimizations, and systematically apply these lessons to future tasks

References

Matrix Mantra (Pvt) Ltd, Matrix Mantra Official Corporate Website. [Online]. Available: <https://matrixmantra.com>.